

HelixCode

HelixCode

Tagline: The distributed AI development platform that divides work, preserves it, and never loses your place.

Summary HelixCode is an enterprise-grade, Go-based distributed AI development platform that splits development work into intelligently-divided tasks across a network of SSH-managed workers, with automatic checkpointing and rollback so no work is ever lost. It unifies multi-provider LLM integration, full development-lifecycle workflows, and cross-platform delivery behind REST, CLI, TUI, and MCP interfaces.

Short description HelixCode is a distributed AI development platform written in Go. It divides work into intelligent tasks across SSH-based worker networks, preserves progress with automatic checkpointing and rollback, integrates multiple LLM providers, and drives the full development lifecycle through REST, CLI, TUI, and MCP interfaces.

Long description HelixCode is an enterprise-grade distributed AI development platform (`dev.helix.code`, MIT) designed for intelligent task division, work preservation, and cross-platform development workflows. Built in Go for scalability, it provides a foundation for distributed computing with automatic checkpointing, rollback, and real-time monitoring.

Its architecture layers a REST + WebSocket + MCP API surface over core services — JWT authentication and session management, SSH-based worker-pool management with health monitoring, task management with checkpointing and dependency handling, project and workflow management, and a unified LLM provider layer — persisted on PostgreSQL (with optional Redis). Distributed workers auto-install across a network, and multi-client interfaces span CLI, terminal UI, REST, and mobile frameworks.

HelixCode drives a complete development lifecycle: planning, building, testing, and refactoring workflows execute automatically with dependency awareness and multi-session context tracking. It integrates multiple LLM providers (Llama.cpp, Ollama, OpenAI) behind one interface, adds hardware-aware model selection (CPU/

GPU/memory detection), and supports advanced reasoning strategies such as chain-of-thought and tree-of-thoughts. The Model Context Protocol is implemented across multiple transports, and multi-channel notifications (Slack, Discord, Email, Telegram) keep teams informed. It targets Linux, macOS, Windows, Aurora OS, and SymphonyOS.

Why we built it Distributed and AI-assisted development typically loses context and progress when tasks are split across machines or interrupted. HelixCode was built to make task division intelligent and work preservation automatic — so a large development effort can be broken down, distributed across a worker network, checkpointed, and resumed or rolled back without losing state.

Why it's a game-changer It combines three things teams usually stitch together from separate tools: distributed compute (SSH worker networks with auto-installation and health monitoring), AI development assistance (multi-provider LLMs with reasoning and tool calling), and full lifecycle workflow automation — all with database-backed checkpointing so distributed work is durable rather than fragile.

What's innovative - Work preservation via automatic checkpointing and rollback applied to distributed development tasks. - Hardware-aware model selection that matches models to detected CPU/GPU/memory capabilities. - A single platform exposing REST, WebSocket, CLI, TUI, and MCP, plus multi-transport MCP support. - Cross-platform reach extending to Aurora OS and SymphonyOS beyond the usual desktop trio.

Biggest technical challenges + how solved - *Not losing work across distributed, interruptible tasks* — solved with a task model carrying checkpoints and dependencies, persisted in PostgreSQL, enabling rollback and resumption. - *Managing a heterogeneous worker fleet* — solved with SSH-based worker registration, auto-installation, and health monitoring in a dedicated worker-pool service. - *Provider and hardware heterogeneity* — solved with a unified LLM provider interface plus hardware detection driving intelligent model selection.

Tech stack - **Go (1.26+ inner module)**: Chosen for concurrency and single-binary distribution suited to distributed workers; used across all core services and CLI/server binaries. - **Gin (HTTP framework)**: Chosen for a fast, minimal REST layer; used to serve the /api/v1 endpoints (auth, workers, tasks, projects). - **PostgreSQL 15+ (via pgx/v5)**: Chosen as the durable system of record; used for the 11-table distributed-computing schema (users, workers, tasks, projects, sessions, llm_providers, notifications). - **Redis 7+ (optional, go-redis/v9)**: Chosen for caching/coordination; used as an optional performance layer. - **SSH**: Chosen as a ubiquitous, secure transport for worker control;

used for worker registration, auto-installation, and command execution across the pool. - **Model Context Protocol (MCP)**: Chosen for standardized tool/context exchange; implemented with multi-transport support. - **LLM providers (Llama.cpp, Ollama, OpenAI)**: Chosen to span local and hosted inference; used behind a unified provider interface with hardware-aware selection.

Public links - Repo: <https://github.com/HelixDevelopment/HelixCode> (public; GitHub description “AI Coding Agent”)

Suggested diagrams/illustrations 1. Layered architecture diagram: API layer (REST/WebSocket/MCP) over core services (auth, worker pool, task/checkpointing, project/workflow, LLM) over the PostgreSQL + Redis data layer. 2. Distributed worker topology: a HelixCode server orchestrating SSH-connected workers across Linux/macOS/Windows/Aurora/SymphonyOS, with health-monitoring indicators. 3. Task lifecycle / work-preservation flow: task division → distributed execution → checkpoint → rollback/resume, shown as a timeline. 4. Development workflow pipeline: planning → building → testing → refactoring with dependency arrows and multi-session context.

Site relevance Both — company framing on vasic.digital (enterprise distributed-AI platform) and personal authorship framing on milosvasic.ru (Milos as architect of the Helix cluster).

Priority tier: Helix-primary

Source provenance - /Volumes/T7/Projects/helix_code/README.md (primary; version, architecture, features, tech stack, endpoints, schema, cross-platform targets, Go/PostgreSQL/Redis versions). - /Volumes/T7/Projects/vasic/_analysis/github-helix-others.md and .json (repo metadata: “AI Coding Agent” description, Go, public, largest repo in fleet). - Repo URL confirmed via README clone URL and harvested inventory. - UNVERIFIED: none of the specific claims above are invented; all trace to the README. Marketing phrasings (“game-changer”, taglines) are editorial, not source metrics. The README’s “FULLY COMPLETE / all 5 phases” status is a self-report — treat completeness claims as project-stated, not independently verified.